

โครงสร้างโปรเจค E-CMIS

คู่มือสำหรับนักพัฒนา (Developer Guide)

ecmis-frontend | ecmis-backend | API Gateway | Database

SS CONSORTIUM — สำนักงาน ปปท.

v1.0 | 30 มีนาคม 2569 | ฉบับเฉพาะทีมพัฒนา

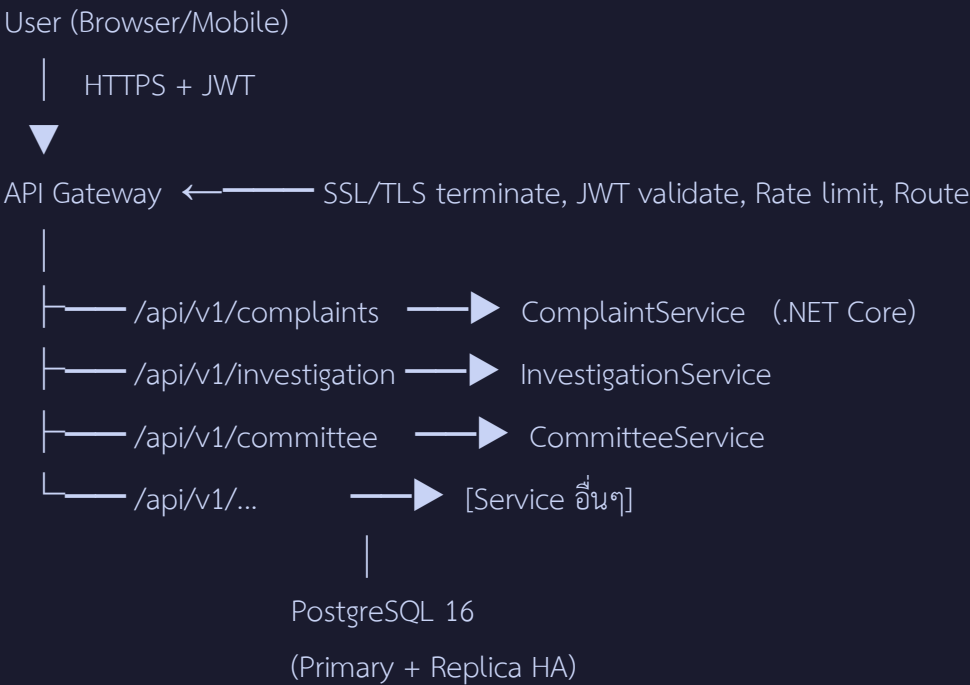
1. ภาพรวมโครงสร้างทั้งระบบ

ระบบ E-CMIS แบ่ง codebase ออกเป็น 2 Repository หลัก บวก infrastructure config แยกต่างหาก ทีมพัฒนาแต่ละส่วนทำงานอิสระจากกัน Deploy ได้แยกกัน และสื่อสารกันผ่าน REST API + JSON เท่านั้น

Repository / Component	Technology	หน้าที่หลัก	ทีมรับผิดชอบ
ecmis-frontend	.NET Blazor + Flutter	UI ทุกหน้าจอ — Web + Mobile	ทีม Frontend
ecmis-backend	.NET Core 8 API	Business Logic + DB + Integrations	ทีม Backend
API Gateway (config)	Kong / Ocelot	Auth, Rate limit, SSL, Routing	ทีม DevOps
infrastructure/	Docker + K8s	Deploy, Scaling, Monitoring	ทีม DevOps

1.1 Flow การทำงาน User → System

Request Flow



1.2 Git Branching Strategy

ทั้ง 2 repo ใช้ Git Flow เดียวกัน ทุก branch มี pipeline ของตัวเอง

Branch	สร้างจาก	Merge กลับ	Auto Deploy
main	—	—	Production (manual approval)
develop	main	main	Dev environment (auto)
feature/xxx	develop	develop	ไม่ deploy อัตโนมัติ
hotfix/xxx	main	main + develop	Production (fast track)
release/x.x	develop	main + develop	Staging environment

2. ecmis-frontend — โครงสร้างทั้งหมด

Frontend repo มี 2 sub-project แยกกันชัดเจน แต่อยู่ใน repo เดียวเพื่อใช้ API contract และ shared constants ร่วมกัน

2.1 Root Structure

Path	ประเภท	คำอธิบาย
ecmis-frontend/	Root	Frontend repository
 web/	Sub-project	.NET Blazor — Web Application
 mobile/	Sub-project	Flutter — iOS + Android
 shared/	Shared	API contract + constants ใช้ร่วมกัน
 .gitlab-ci.yml	CI/CD	Pipeline หลักของ Frontend repo
 .gitignore	Config	ไฟล์ที่ไม่ต้อง commit
 README.md	Docs	วิธีติดตั้งและรัน local

2.2 web/ — .NET Blazor (Web Application)

Web Application สำหรับเจ้าหน้าที่ ปปท. ใช้ผ่าน Browser ทุกหน้าเขียนด้วย Razor Component (.razor) ใช้ C# ทั้งหมด ไม่มี JavaScript

Path	ประเภท	คำอธิบาย
web/	Root	Blazor WebAssembly project
— src/	Source	โค้ดทั้งหมด
— Pages/	Razor Pages	หน้าจอแต่ละโมดูล
— Complaint/	Module	กิจกรรม 4 — รับเรื่องร้องเรียน 15 ช่องทาง
— Investigation/	Module	กิจกรรม 5 — ดำเนินงานกล่าวหา
— WitnessProtect/	Module	กิจกรรม 6 — คุ่มครองพยาน
— Committee/	Module	กิจกรรม 7 — พิจารณา/มติ คณะกรรมการ ปปท.
— PersonCheck/	Module	กิจกรรม 8 — ตรวจสอบประวัติบุคคล
— ArrestWarrant/	Module	กิจกรรม 9 — ตามหมายจับ
— LegalCase/	Module	กิจกรรม 10 — กฎหมายทางคดี
— Analytics/	Module	กิจกรรม 12 — วิเคราะห์สถิติ 12 มิติ
— Admin/	Module	กิจกรรม 14 — User / Role / Organization
— Public/	Module	กิจกรรม 14 — ประชาสัมพันธ์/เว็บไซต์
— Components/	Shared UI	Component ใช้ร่วมทุกหน้า
— Layout/	Layout	MainLayout, NavMenu, SideBar, Header
— DataGrid/	Table	ตารางข้อมูล + Sort + Filter + Paginate
— Dashboard/	Chart	กราฟ Real-Time ด้วย SignalR
— FileUpload/	Upload	อัปโหลด Word/Excel/PDF
— Modal/	Dialog	Confirm, Alert, Error dialog
— Services/	API Clients	เรียก Backend Microservice แต่ละตัว
— ComplaintService.cs	HTTP Service	เรียก /api/v1/complaints
— InvestigationService.cs	HTTP Service	เรียก /api/v1/investigations
— AuthService.cs	Auth	JWT + ThaiID login/logout/refresh

└── [ServiceName]Service.cs	HTTP Service	1 ไฟล์ต่อ 1 Microservice
└── Models/	DTOs	C# class รับ-ส่งข้อมูลกับ API
└── wwwroot/	Static files	CSS, JS, font, รูปภาพ
└── Program.cs	Entry point	Register all services + middleware
└── tests/	Test	Automated tests
└── unit/	bUnit	ทดสอบ Blazor Component แต่ละตัว
└── e2e/	Selenium	ทดสอบ flow ทั้งหมดผ่าน Browser
└── Dockerfile	Docker	Build Docker image สำหรับ deploy

โครงสร้างภายใน Module (ตัวอย่าง Complaint/)

Path	ประเภท	คำอธิบาย
Complaint/	Module root	
└── ComplaintList.razor	Page	รายการ + Dashboard + Export
└── ComplaintList.razor.cs	Code-behind	C# logic แยกไฟล์
└── ComplaintCreate.razor	Page	Form สร้างเรื่องร้องเรียน + 15 ช่องทาง
└── ComplaintDetail.razor	Page	รายละเอียด + Timeline + ไฟล์แนบ
└── Components/	Local UI	Component เฉพาะ Module นี้
└── ComplaintCard.razor	Card	แสดง summary 1 เรื่อง
└── StatusBadge.razor	Badge	สถานะพร้อมสี
└── ChannelSelector.razor	Input	Dropdown 15 ช่องทาง
└── Models/	Local DTOs	DTO เฉพาะ Module นี้

2.3 mobile/ — Flutter (iOS + Android)

Mobile Application สำหรับผู้บริหารและนักสืบ ใช้ Dart เขียน codebase เดียวได้ทั้ง iOS และ Android ใช้ Riverpod เป็น State Management และ Dio เป็น HTTP Client

Path	ประเภท	คำอธิบาย
mobile/	Root	Flutter project
├── lib/	Source	โค้ดทั้งหมด
│ ├── features/	Features	แยกตาม Feature ของ App
│ │ ├── dashboard/	Feature	Dashboard ผู้บริหาร — KPI, กราฟ, สรุป
│ │ ├── complaint/	Feature	ดูสถานะเรื่องร้องเรียน
│ │ ├── case_tracking/	Feature	ติดตามสำนวนคดี
│ │ ├── investigation/	Feature	กระบวนการไต่สวน
│ │ ├── warrant/	Feature	ตามหมายจับ
│ │ └── notification/	Feature	Push notification + Line alert
│ └── core/	Core Layer	ของใช้ร่วมทุก Feature
│ ├── api/	HTTP	Dio client + interceptors
│ ├── auth/	Auth	ThaiD OAuth2 + JWT + SecureStorage
│ ├── router/	Navigation	GoRouter — page routing
│ └── utils/	Utilities	Date, Thai text, formatter, validators
│ └── shared/	Shared UI	Widget ใช้ร่วมทุก Feature
│ ├── widgets/	Components	Button, Card, Badge, Loading, Dialog
│ ├── theme/	Design System	สี, typography, spacing
│ └── l10n/	Localization	ภาษาไทย / English
└── test/	Test	Flutter tests
│ ├── unit/	Unit	ทดสอบ Logic, Repository, Provider
│ ├── widget/	Widget	ทดสอบ UI Component
│ └── integration/	E2E	ทดสอบ flow ทั้งหมดบน device/emulator
└── android/	Android	Gradle config, signing keystore
└── ios/	iOS	Xcode project, Podfile, signing

└─ assets/	Assets	รูป, icon, font
└─ pubspec.yaml	Config	Dependencies ทั้งหมด
└─ Dockerfile	Docker	Build image สำหรับ CI

โครงสร้างภายใน Feature (ตัวอย่าง complaint/)

Path	ประเภท	คำอธิบาย
complaint/	Feature root	
└─ data/	Data Layer	ดึงข้อมูลจาก API
└─ complaint_api.dart	API	HTTP calls ไปที่ Backend
└─ complaint_repository.dart	Repository	แปลง JSON → Dart model
└─ domain/	Domain Layer	Business model
└─ complaint_model.dart	Model	ข้อมูลเรื่องร้องเรียน (Freezed)
└─ complaint_status.dart	Enum	สถานะคดีทุกประเภท
└─ presentation/	UI Layer	หน้าจอ
└─ complaint_list_page.dart	Page	รายการเรื่องร้องเรียน
└─ complaint_detail_page.dart	Page	รายละเอียด
└─ widgets/	UI Widgets	Card, StatusChip, Timeline
└─ state/	State Layer	จัดการ state ด้วย Riverpod
└─ complaint_provider.dart	Provider	โหลด + Cache ข้อมูล
└─ complaint_notifier.dart	Notifier	อัปเดต state

2.4 Package หลัก

Platform	Package	Version	หน้าที่
Blazor	Microsoft.AspNetCore.Components.WebAssembly	8.x	Core framework
Blazor	MudBlazor	6.x	UI Components — DataGrid, Chart, Dialog
Blazor	Microsoft.AspNetCore.SignalR.Client	8.x	Real-time Dashboard
Blazor	FluentValidation	11.x	Form validation
Blazor	Blazored.LocalStorage	4.x	JWT token storage
Flutter	dio	^5.x	HTTP Client เรียก Backend
Flutter	flutter_riverpod	^2.x	State Management
Flutter	go_router	^13.x	Page navigation
Flutter	freezed	^2.x	Immutable data models
Flutter	flutter_secure_storage	^9.x	เก็บ Token อย่างปลอดภัย
Flutter	fl_chart	^0.68	กราฟ Dashboard

3. ecmis-backend — โครงสร้างทั้งหมด

Backend repo รับผิดชอบ Business Logic ทั้งหมด ออกแบบเป็น Microservice Architecture แต่ละ Service รับผิดชอบ 1 Domain สามารถ Build, Test และ Deploy ได้อิสระ ทุก Service ใช้ PostgreSQL เป็นฐานข้อมูลหลัก

3.1 Root Structure

Path	ประเภท	คำอธิบาย
ecmis-backend/	Root	Backend repository
├── services/	Microservices	14 Service ตาม TOR กิจกรรม 4-14
├── shared/	Shared Library	ใช้ร่วมทุก Service
├── integrations/	API Connectors	เชื่อมต่อหน่วยงานภายนอก
├── database/	DB Scripts	Migration, Seed, Data Migration
├── infrastructure/	IaC	Docker, K8s, CI/CD config
├── .gitlab-ci.yml	CI/CD	Pipeline หลัก
└── README.md	Docs	Getting started guide

3.2 services/ — Microservice ทั้ง 14 ตัว

แต่ละ Service อยู่ใน folder ของตัวเอง Deploy แยกกันเป็น Docker Container ใน Kubernetes

Service Name	TOR กิจกรรม	หน้าที่
ComplaintService	4	รับเรื่องร้องเรียน 15 ช่องทาง ออกเลขคดีอัตโนมัติ
InvestigationService	5	กระบวนการดำเนินงานกล่าวหา — ไต่สวนทุกขั้นตอน
WitnessService	6	คุ้มครองพยาน — รับคำร้อง พิจารณา ติดตาม
CommitteeService	7	พิจารณา/มติคณะกรรมการ ปปท.
PersonCheckService	8	ตรวจสอบประวัติบุคคล + ติดตามภายหลัง ปปท. มีมติ
ArrestWarrantService	9	ฐานข้อมูลตามหมายจับ — บันทึก ตรวจสอบ ติดตามสถานะ
LegalService	10	กระบวนการด้านกฎหมาย — ขั้นตอนการและชั้นศาล
MigrationService	11	Data Migration — ย้ายข้อมูลจากระบบเดิม
AnalyticsService	12	วิเคราะห์สถิติ 12 มิติ + รายงาน + BI Dashboard
IntegrationService	13	API เชื่อมโยงหน่วยงานภายนอกทุกตัว
AdminService	14	User, Role, Organization, Log, Digital Signature
NotificationService	13	แจ้งเตือน Line / Email อัตโนมัติ
WorkflowService	5,7	Workflow Engine — State Machine กระบวนการพิจารณา
ReportService	12	Export Word/Excel/PDF + BI

3.3 โครงสร้างภายในแต่ละ Service (Clean Architecture)

ทุก Service ใช้โครงสร้างเดียวกัน แบ่งเป็น 4 Layer เพื่อให้แก้ไข Business Logic ได้โดยไม่กระทบ Infrastructure

Path	Layer	คำอธิบาย
ComplaintService/	Root	ตัวอย่าง Service
├── Controllers/	Layer 1: API	รับ HTTP Request — บางที่สุด ไม่มี Logic
│ └── ComplaintsController.cs	Controller	GET/POST/PUT/DELETE endpoints
├── Services/	Layer 2: Business	Business Logic ทั้งหมดอยู่ที่นี้
│ ├── IComplaintService.cs	Interface	กำหนด Contract ของ Service
│ └── ComplaintService.cs	Implementation	Logic จริง: ออกเลขคดี, Validate, Workflow
├── Repositories/	Layer 3: Data	คุยกับ Database อย่างเดียว
│ ├── IComplaintRepository.cs	Interface	CRUD Contract
│ └── ComplaintRepository.cs	EF Core	Query PostgreSQL
├── Models/	Domain + DTOs	Data structures
│ ├── Entities/Complaint.cs	Entity	ตาราง complaints ใน DB
│ ├── DTOs/CreateComplaintDto.cs	Input DTO	รับข้อมูลจาก Client
│ └── DTOs/ComplaintResponseDto.cs	Output DTO	ส่งข้อมูลกลับ Client
├── Events/	Domain Events	แจ้ง Service อื่นเมื่อ State เปลี่ยน
│ └── ComplaintCreatedEvent.cs	Event	ส่งต่อไป NotificationService
├── Validators/	Validation	ตรวจสอบ Input ก่อนเข้า Logic
│ └── CreateComplaintValidator.cs	FluentValidation	required, format, business rules
├── Mappings/	AutoMapper	แปลง Entity ↔ DTO
├── Program.cs	Entry Point	Register services, middleware, Swagger
├── appsettings.json	Config	Connection strings (env override ใน prod)
├── Dockerfile	Container	Build Docker image
└── Tests/	Test	xUnit Unit + Integration tests

3.4 shared/ — Library ใช้ร่วมทุก Service

ของที่ทุก Service ต้องการเหมือนกัน รวมไว้ที่นี่ เพื่อไม่ให้เขียนโค้ดซ้ำ

Path	ประเภท	คำอธิบาย
shared/	Root	Shared class library (.NET)
— Domain/	Domain Models	Entities, Enums, Value Objects ใช้ร่วม
— Entities/BaseEntity.cs	Base	Id, CreatedAt, UpdatedAt ทุก entity มี
— Enums/ComplaintStatus.cs	Enum	สถานะที่ใช้ร่วมกันหลาย Service
— ValueObjects/	VO	CaseNumber, ThaiDate ฯลฯ
— Infrastructure/	EF Core + Utils	ของใช้ร่วมด้าน Infrastructure
— EcmisDbContext.cs	DbContext	EF Core context หลักของระบบ
— RepositoryBase.cs	Base Repo	CRUD generic ทุก Repository extends นี้
— RedisService.cs	Cache	Redis helper — cache, session, rate limit
— StorageService.cs	File	S3-compatible storage สำหรับเก็บไฟล์แนบ
— Common/	Utilities	Helper, DTO, Exception ทั่วไป
— DTOs/PagedResult.cs	Pagination	Pagination wrapper ใช้ทุก list endpoint
— Exceptions/	Exceptions	NotFoundException, ForbiddenException ฯลฯ
— Middleware/	Middleware	Error handler, Request logging, CORS
— Extensions/	Extensions	Extension methods ที่ใช้บ่อย
— Auth/	Security	JWT + Permission ใช้ร่วม
— JwtHelper.cs	JWT	สร้าง / Validate JWT token
— PermissionConstants.cs	Permissions	ชื่อ Permission ทุกตัวของระบบ
— AuthorizationPolicies.cs	Policies	Role-based + Permission-based policy

3.5 database/ — PostgreSQL

Path	ประเภท	คำอธิบาย
database/	Root	Database scripts ทั้งหมด
├── migrations/	EF Migrations	Schema version control — อย่าแก้ไขไฟล์เดิม
├── 20250101_InitialCreate.cs	Migration	สร้าง schema ครั้งแรก
└── 20250201_AddCaseIndex.cs	Migration	แต่ละ migration คือ 1 การเปลี่ยนแปลง
├── seeds/	Seed Data	ข้อมูลตั้งต้นที่ต้องมีก่อน go-live
├── 01_offices.sql	Seed	สำนักงาน ปปท. ส่วนกลาง + เขต 1-9
├── 02_complaint_types.sql	Seed	ประเภทเรื่องร้องเรียน
├── 03_channels.sql	Seed	15 ช่องทางรับเรื่อง
└── 04_roles_permissions.sql	Seed	Role และ Permission เริ่มต้น
├── schemas/	Documentation	ERD และ Data Dictionary
├── erd.md	ERD	Mermaid ERD diagram
└── data_dictionary.xlsx	Dict	คำอธิบายทุก table/column
└── data-migration/	ETL	กิจกรรม 11 — ย้ายข้อมูลจากระบบเดิม
├── strategy.md	Plan	Data Migration Strategy
├── mapping/	Mapping	Source → Target field mapping
├── cleaning/	Scripts	Data cleaning + validation scripts
└── dry-run/	Test	Script สำหรับ Dry Run ≥ 2 รอบ

3.6 integrations/ — External API Connectors

กิจกรรม 13 — เชื่อมโยงหน่วยงานภายนอก แต่ละ connector อยู่ใน folder ของตัวเอง มี Interface เดียวกัน

Connector	Protocol	หน่วยงาน	ข้อมูลที่เชื่อมโยง
ThaiID	OAuth 2.0	กรมการปกครอง	ยืนยันตัวตนดิจิทัล บัตร ปชช.
DPIS	REST API	กรมการปกครอง	ข้อมูลบุคลากรภาครัฐ
DOL	REST API	กรมการปกครอง	ทะเบียนราษฎร์
DLT	REST API	กรมขนส่งทางบก	ข้อมูลรถและผู้ขับขี่
SSO	REST API	ประกันสังคม	ข้อมูลนายจ้าง-ลูกจ้าง
NACC	REST API	ป.ป.ช.	รับ-ส่งข้อมูลคดีทุจริต
AWIS	SOAP + REST	ศาลยุติธรรม	หมายออก, คำพิพากษา
CitizenPortal	REST API	สพร. (แอปทางรัฐ)	รับเรื่องร้องเรียนจากประชาชน
LineNotify	REST API	LINE Corp.	แจ้งเตือนผ่าน Line Application
EmailSMTP	SMTP	SMTP Server ปปท.	แจ้งเตือนทางอีเมล
NSTP	REST API	กพร.	ระบบ ศปท. — เรื่องร้องเรียน

4. Database Schema — PostgreSQL

ใช้ PostgreSQL 16 เป็นฐานข้อมูลหลัก ออกแบบเป็น Single database หลายหน่วยงาน ใช้ row-level security แยก data ตาม office_id

4.1 Core Tables หลัก

Table Name	Schema	Primary Key	คำอธิบาย
complaints	public	uuid	เรื่องร้องเรียนทั้งหมด — ตารางหลักของระบบ
investigations	public	uuid	สำนวนคดีและการไต่สวน
committee_resolutions	public	uuid	มติคณะกรรมการ ปปท.
witness_protections	public	uuid	คำร้องขอคุ้มครองพยาน
arrest_warrants	public	uuid	หมายจับ + ติดตามสถานะ
legal_cases	public	uuid	คดีในชั้นอุทธรณ์และศาล
persons	public	uuid	บุคคล ผู้กล่าวหา/ถูกกล่าวหา
users	auth	uuid	ผู้ใช้งานระบบ
roles	auth	uuid	Role การใช้งาน
permissions	auth	uuid	Permission แต่ละ action
offices	master	uuid	สำนักงาน ปปท. ส่วนกลาง + เขต 1-9
audit_logs	audit	bigint	บันทึกทุก action ของผู้ใช้

4.2 ตัวอย่าง Schema: complaints table

complaints table — PostgreSQL

```
CREATE TABLE public.complaints (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  case_number VARCHAR(30) NOT NULL UNIQUE, -- ปปท.-01-2568-00001  
  office_id   UUID          NOT NULL REFERENCES master.offices(id),  
  subject     TEXT          NOT NULL,  
  channel     VARCHAR(50) NOT NULL, -- website, email, walk_in, 1206...  
  complaint_type VARCHAR(100),  
  incident_date DATE,  
  location_province VARCHAR(100),  
  location_district VARCHAR(100),  
  accused_agency VARCHAR(200),  
  accused_name  TEXT,  
  accuser_name  TEXT,  
  accuser_phone VARCHAR(20),  
  status        VARCHAR(50) NOT NULL DEFAULT 'received',  
  assigned_to   UUID          REFERENCES auth.users(id),  
  priority      SMALLINT NOT NULL DEFAULT 3, -- 1=urgent, 5=normal  
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by    UUID          NOT NULL REFERENCES auth.users(id)  
);  
  
-- Index สำหรับ search บ่อย  
CREATE INDEX idx_complaints_status ON complaints(status);  
CREATE INDEX idx_complaints_office ON complaints(office_id);  
CREATE INDEX idx_complaints_case_number ON complaints(case_number);  
CREATE INDEX idx_complaints_created_at ON complaints(created_at DESC);
```

4.3 กฎ Database ที่ Dev ต้องปฏิบัติ

- Primary Key ทุกตัวเป็น UUID ไม่ใช่ int — ป้องกัน sequential guess
- ทุก table ต้องมี created_at, updated_at, created_by เสมอ
- ห้าม Hard Delete — ใช้ soft delete ด้วย deleted_at TIMESTAMPTZ
- Migration ใหม่ทุกครั้งต้องมีทั้ง Up และ Down script
- ชื่อ column ใช้ snake_case เสมอ เช่น case_number ไม่ใช่ caseNumber
- Enum ที่อาจเพิ่มค่าในอนาคต ให้เก็บเป็น VARCHAR ไม่ใช่ PostgreSQL ENUM type

5. API Design Standards

ทุก Microservice ใช้ REST API ตามมาตรฐานเดียวกัน เพื่อให้ Frontend ทั้ง Blazor และ Flutter ทำงานร่วมได้โดยไม่ต้องเรียนรู้ API ใหม่ทุกครั้ง

5.1 URL Convention

URL Pattern

Base URL: `https://api.ecmis.go.th/v1`

GET `/complaints` # รายการทั้งหมด (พร้อม pagination)

GET `/complaints/{id}` # รายละเอียด 1 รายการ

POST `/complaints` # สร้างใหม่

PUT `/complaints/{id}` # อัปเดตทั้งหมด

PATCH `/complaints/{id}/status` # อัปเดตบางส่วน

DELETE `/complaints/{id}` # soft delete (ตั้ง `deleted_at`)

Nested resource

GET `/complaints/{id}/attachments` # ไฟล์แนบของเรื่อง

POST `/complaints/{id}/attachments` # อัปโหลดไฟล์แนบ

Query parameters

GET `/complaints?status=received&office=01&page=1&limit=20&sort=created_at:desc`

5.2 Response Format มาตรฐาน

Standard Response (ทุก endpoint ตอบแบบนี้)

```
// Success — Single item
{
  "success": true,
  "data": { "id": "uuid", "caseNumber": "ปปท.-01-2568-00001", ... },
  "meta": null
}

// Success — List with pagination
{
  "success": true,
  "data": [ {...}, {...} ],
  "meta": { "page": 1, "limit": 20, "total": 150, "totalPages": 8 }
}

// Error
{
  "success": false,
  "error": {
    "code": "COMPLAINT_NOT_FOUND",
    "message": "ไม่พบเรื่องร้องเรียนเลขที่ระบุ",
    "details": null
  }
}
```

5.3 HTTP Status Codes ที่ใช้

Code	ความหมาย	ใช้เมื่อ
200	OK	GET สำเร็จ, PUT/PATCH สำเร็จ
201	Created	POST สร้างข้อมูลใหม่สำเร็จ
204	No Content	DELETE สำเร็จ (ไม่มี body)
400	Bad Request	ข้อมูลที่ส่งมาไม่ถูกต้อง (Validation error)
401	Unauthorized	ไม่มี JWT Token หรือ Token หมดอายุ
403	Forbidden	มี Token แต่ไม่มีสิทธิ์ทำ action นี้
404	Not Found	ไม่พบข้อมูลที่ request
422	Unprocessable	Business rule ไม่ผ่าน เช่น สถานะเปลี่ยนไม่ได้
500	Server Error	เกิด exception ที่ไม่ได้คาดไว้ — ต้อง log ทุกครั้ง

5.4 Authentication — JWT

JWT Token Flow

```
// 1. Login → ได้ Access Token + Refresh Token
POST /auth/login
{ "username": "...", "password": "..." }
→ { "accessToken": "eyJ...", "refreshToken": "...", "expiresIn": 3600 }

// 2. ทุก request ส่ง Access Token ใน Header
Authorization: Bearer eyJ...

// 3. เมื่อ Access Token หมดอายุ (401) → Refresh
POST /auth/refresh
{ "refreshToken": "..." }
→ { "accessToken": "eyJ...(ใหม่)", "expiresIn": 3600 }

// Token Expiry
Access Token : 1 ชั่วโมง
Refresh Token : 7 วัน
```

6. DevSecOps — CI/CD Pipeline

ทั้ง 2 repo มี .gitlab-ci.yml ของตัวเอง ทุก Push จะ trigger Pipeline อัตโนมัติ Security scan รันใน Pipeline ทุกครั้งก่อน Deploy

6.1 Pipeline Stages ทั้ง 2 Repo

#	Stage	Tool	ทำอะไร
1	build	Docker	Compile + Build Docker image
2	test	xUnit / flutter_test	Unit + Integration test (coverage ≥ 80%)
3	lint	SonarQube	Code quality + code smell + duplication
4	sast	OWASP Dependency Check	ช่องโหว่ใน dependencies (ทุก branch)
5	deploy-dev	Kubernetes	Deploy ไป Dev environment (branch: develop)
6	dast	OWASP ZAP	Dynamic scan — ยิง test จริงบน staging
7	pen-test	Manual + Burp Suite	ทดสอบก่อน release ทุกครั้ง
8	deploy-prod	Kubernetes (manual)	Deploy Production — ต้องมีคนกด Approve

6.2 Infrastructure — Docker + Kubernetes

Path	ประเภท	คำอธิบาย
infrastructure/	Root	laC ทั้งหมด
└─ docker/	Docker	Container definitions
└─ docker-compose.dev.yml	Compose	รัน local development (ทุก service พร้อมกัน)
└─ docker-compose.test.yml	Compose	รัน integration test
└─ .env.example	Template	ตัวอย่าง environment variables
└─ k8s/	Kubernetes	Production deployment
└─ namespaces/	NS	ecmis-dev, ecmis-staging, ecmis-prod
└─ deployments/	Deploy	1 ไฟล์ต่อ 1 Service
└─ services/	Service	ClusterIP / LoadBalancer per service
└─ ingress/	Ingress	API Gateway routing rules
└─ configmaps/	Config	non-secret config per environment
└─ secrets/	Secrets	DB password, API keys (sealed secrets)
└─ monitoring/	Observability	Prometheus + Grafana + Loki
└─ prometheus.yml	Metrics	Scrape config ทุก service
└─ grafana/	Dashboard	Dashboard templates
└─ alerts/	Alerts	Alert rules (down, high error rate, slow API)

6.3 Environment Variables

ห้าม commit ค่า secret ลงใน Git เด็ดขาด — ใช้ GitLab CI/CD Variables หรือ K8s Secrets เท่านั้น

appsettings.json (ค่าที่ safe commit ได้)

```
{
  "Logging": { "LogLevel": { "Default": "Information" } },
  "AllowedHosts": "*",
  "Swagger": { "Enabled": true }
}

# Environment variables ที่ inject ตอน runtime (ห้ามอยู่ใน appsettings)
ECMIS_DB_CONNECTION=Host=db;Database=ecmis;Username=...;Password=...
ECMIS_JWT_SECRET=...
ECMIS_THAI_D_CLIENT_ID=...
ECMIS_THAI_D_CLIENT_SECRET=...
ECMIS_LINE_NOTIFY_TOKEN=...
ECMIS_REDIS_CONNECTION=redis:6379
```

7. Coding Standards & Conventions

ข้อตกลงที่ Dev ทุกคนในทีมต้องปฏิบัติตาม เพื่อให้โค้ดอ่านง่ายและ maintain ได้ในระยะยาว

7.1 Naming Convention

ภาษา / Layer	Pattern	ตัวอย่าง WRONG	ตัวอย่าง CORRECT
C# Class	PascalCase	complaintService	ComplaintService
C# Method	PascalCase	getAll()	GetAllAsync()
C# Property	PascalCase	case_number	CaseNumber
C# Variable	camelCase	CaseNumber	caseNumber
C# Interface	IPascalCase	ComplaintService	IComplaintService
Dart Class	PascalCase	complaint_service	ComplaintService
Dart Method	camelCase	GetAll()	getAll()
Dart File	snake_case	ComplaintService.dart	complaint_service.dart
DB Table	snake_case	ComplaintTable	complaints
DB Column	snake_case	CaseNumber	case_number
API Endpoint	kebab-case	/getComplaints	/complaints
API JSON Field	camelCase	case_number	caseNumber

7.2 Code Quality Rules

- Unit test coverage ต้องไม่ต่ำกว่า 80% — Pipeline จะ fail ถ้าต่ำกว่านี้
- ทุก public method ต้องมี XML doc comment (C#) หรือ doc comment (Dart)
- ไม่มี magic number ในโค้ด — ต้องนิยามเป็น constant หรือ enum
- ทุก async method ต้องมี suffix Async เช่น GetAllAsync()
- Error handling: ทุก Service method ต้องมี try-catch และ log error
- ห้าม return null จาก Service — ใช้ NotFoundException แทน
- ทุก Database query ต้องผ่าน Repository เท่านั้น — ห้าม query ตรงใน Controller หรือ Service

7.3 Commit Message Convention

Conventional Commits Format

<type>(<scope>): <description>

type:

- feat — เพิ่ม feature ใหม่
- fix — แก้ bug
- refactor — refactor โค้ด (ไม่มี feature ใหม่ ไม่ได้แก้ bug)
- test — เพิ่ม/แก้ test
- docs — แก้ documentation
- ci — แก้ CI/CD config
- chore — งานอื่นๆ ที่ไม่กระทบโค้ด

ตัวอย่าง:

- feat(complaint): add auto case number generation
- fix(auth): fix JWT token not refreshing on 401
- test(investigation): add unit tests for workflow state machine
- docs(readme): update local setup instructions

7.4 Pull Request Checklist

ก่อน create PR ทุกครั้ง Dev ต้องตรวจสอบ:

- Pipeline สีเขียวทั้งหมด (build, test, lint ผ่าน)
- Test coverage ไม่ลด — ถ้าเพิ่มโค้ดใหม่ต้องเพิ่ม test ด้วย
- ไม่มี console.log หรือ Debug.WriteLine ค้างในโค้ด
- Migration ใหม่มีทั้ง Up และ Down script
- API ใหม่มี Swagger annotation ครบ
- Secret ไม่มีใน code หรือ config file
- PR description อธิบายว่าทำอะไร และ test ยังไง
- Reviewers อย่างน้อย 1 คน approve ก่อน merge

8. Local Development Setup

ขั้นตอนสำหรับ Dev ที่เพิ่งเริ่มต้น setup เครื่องเพื่อทำงานกับโปรเจกต์นี้

8.1 Prerequisites — ติดตั้งก่อนเริ่ม

Software	Version	Download / หมายเหตุ
Git	≥ 2.40	git-scm.com
Docker Desktop	≥ 4.28	docker.com — ต้องเปิด WSL2 integration (Windows)
.NET SDK	8.0 LTS	dotnet.microsoft.com/download
Node.js	≥ 20 LTS	nodejs.org — สำหรับ tooling
Flutter SDK	≥ 3.19	flutter.dev
Visual Studio / Rider	Latest	สำหรับ .NET development
VS Code	Latest	+ Extensions: Dart, Flutter, C# Dev Kit
DBeaver / pgAdmin	Latest	PostgreSQL GUI client

8.2 ขั้นตอน Setup Backend (ecmis-backend)

Setup Backend

```
# 1. Clone repo
git clone https://gitlab.ecmis.go.th/ss-consortium/ecmis-backend.git
cd ecmis-backend

# 2. รัน PostgreSQL + Redis ด้วย Docker (ไม่ต้องติดตั้ง local)
docker-compose -f infrastructure/docker/docker-compose.dev.yml up -d

# 3. Copy environment config
cp .env.example .env
# แก้ไข .env ตามที่ได้รับจาก Tech Lead

# 4. รัน Database Migration
```

```
dotnet ef database update --project services/ComplaintService
```

```
# 5. รัน Seed data
```

```
dotnet run --project database/seeds
```

```
# 6. รัน Service ที่ต้องการ (ตัวอย่าง ComplaintService)
```

```
dotnet run --project services/ComplaintService
```

```
# Swagger UI: http://localhost:5001/swagger
```

8.3 ขั้นตอน Setup Frontend (ecmis-frontend)

Setup Frontend

```
# 1. Clone repo
```

```
git clone https://gitlab.ecmis.go.th/ss-consortium/ecmis-frontend.git
```

```
cd ecmis-frontend
```

```
# — Web (Blazor) —
```

```
cd web
```

```
dotnet restore src/
```

```
cp src/appsettings.example.json src/appsettings.Development.json
```

```
# แก้ ApiBaseUrl ให้ชี้ไป Backend local
```

```
dotnet run --project src/
```

```
# เปิด Browser: http://localhost:5173
```

```
# — Mobile (Flutter) —
```

```
cd ../mobile
```

```
flutter pub get
```

```
flutter run --flavor dev -t lib/main_dev.dart
```

```
# หรือรันบน iOS Simulator / Android Emulator
```

8.4 Port ที่ใช้ใน Local Development

Service	Port	Protocol	หมายเหตุ
Blazor Web	5173	HTTP	Frontend Web App
API Gateway	8080	HTTP	ชี้ไปทุก service
ComplaintService	5001	HTTP	+ Swagger ที่ /swagger
InvestigationService	5002	HTTP	
CommitteeService	5003	HTTP	
AdminService	5010	HTTP	
PostgreSQL	5432	TCP	user: ecmis, db: ecmis_dev
Redis	6379	TCP	ไม่มี password ใน dev
Prometheus	9090	HTTP	Metrics dashboard
Grafana	3000	HTTP	admin / admin123